Unit 6. Usupervised Learning

Machine Learning 1 (ML1)

Master in Artificial Intelligence (MIA)

November 25, 2025

5.1 Introduction

Unsupervised learning is characterized by the absence of any **desired output** for the system in the **training** dataset.

- No class data in the dataset, **only input features**.
- Labeling instances is impractical (costly or impossible to do).

Unsupervised algorithms have to discover by themselves hidden **patterns**, **regularities**, **correlations** or **groupings** in the input data without the need for human intervention.

- Only possible when there is a degree of **redundancy** in the input data.
- Without redundancy it is impossible to find patterns or groupings.

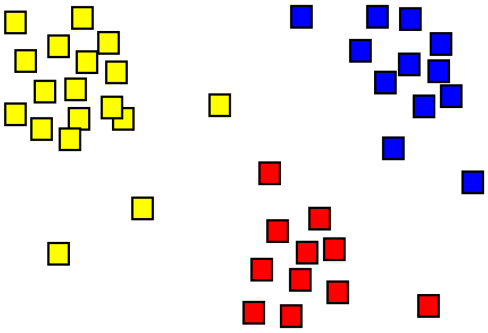
Some applications of unsupervised learning **techniques**:

- **Cluster analysis**: a data mining technique which **groups unlabeled data** based on their mutual similarity or differences.
- **Dimensionality reduction**: given a high dimensional dataset, find a **transformation** into a lower dimensional space keeping some meaningful properties of the original data. E.g., PCA.
- **Data visualization**: given data with a complex representation (highly dimensional) generate a **representation** that humans can comprehend. E.g., t-SNE.
- **Blind signal separation**: is the separation of a set of **source** signals from a set of **mixed** signals, without the aid of information about the source signals or the mixing process. E.g., ICA.
- **Feature extraction**: generation of **new features** from the original ones without human intervention.
- **Association rules**: rule-based methods for finding **relationships** between variables in a given dataset. The rules present how often a certain data item occurs in datasets and how strong and weak the connections between different objects are.

5.2 Cluster Analysis

Cluster analysis, often called **clustering**, is the process of partitioning a set of data objects (instances) into subsets.

- Each subset is a **cluster**.
- Objects in a cluster are **similar to one another**, but **dissimilar** to objects in **other clusters**.
- The set of clusters resulting from a cluster analysis can be referred to as a **clustering** of the original data.
- Different clustering methods may generate different clusterings on the same data set.
  - Unsupervised methods without human intervention.
- **Usefulness**: discovery of previously unknown groups within the data.



Source: wikipedia (Author: Wgabrie)

A cluster of data objects can be treated as an implicit class. In this sense, clustering is sometimes called **automatic classification** or **unsupervised classification**.

Clustering is also called **data segmentation** in some applications because it partitions large data sets into groups according to their similarity.

As mentioned elsewhere in this course clustering can be used in **outlier detection**, **noise reduction** and **discretization**.

Some **requirements** for effective cluster analysis:

- **Scalability**: many clustering algorithms work well on small data sets (hundreds of instances), but real world data is considerably larger.

  Clustering on only a sample of a given large data set may lead to biased results ⇒ scalable algorithms are needed.
- **Ability to deal with different types of attributes**: while most algorithms are designed to cluster numeric data, applications may require clustering other data types, such as binary, nominal (categorical), and ordinal data.
- **Clusters with arbitrary shape**: many algorithms determine clusters based on distance measures, which leads to spherical shapes. However, a cluster could be of any shape.
- **Domain-based input parameters**: many algorithms require users to provide domain knowledge in the form of input parameters (E.g., number of clusters).
  - Clustering results may be sensitive to such parameters.
  - Parameters may be difficult to determine.
- **Ability to deal with noisy data**: most real-world data sets contain outliers and/or missing, unknown, or erroneous data. Clustering methods have to be are robust with respect to such noise.
- **Incremental clustering**: real-world data may arrive periodically in an incremantal fashion. Clustering algorithms need to be able to accept new data without performing a new partition from scratch.
- **Insensitivity to input order**: the output of the algorithm should be insensitive to the order in which process the data, which sometimes is not the case.
- **Dealing with high-dimensionality data**: high dimensional data is inherently sparser than low dimensional data, making the search for clusterings more challenging.
- **Interpretability and usability**: Users want clustering results to be interpretable, comprehensible, and usable. That is, clustering may need to be tied in with specific semantic interpretations and applications.

There is a miriad of clustering methods. We are going to group the most basic approaches into a few classes:

- **Partitioning methods**: Given a set of n objects, a partitioning method constructs k partitions of the data, where each partition represents a cluster and $k \leq n$.

  Each cluster has a least an instance and each instance belongs to a cluster (exclusive cluster separation).

  They use **iterative relocation techniques** that attempt to improve the partitioning by moving objects from one group to another.

  Popular methods: k-means and k-medoids.
- **Hierarchical methods**: creates a hierarchical **decomposition** of the given examples into a **tree** of clusters, in which each parent node has the same instances that its children.

  Popular methods: COBWEB.
- **Density-based methods**: any given cluster is grown as long as the **density** (number of objects or data points) in its neighborhood (defined by a given radius) exceeds some **threshold**.

  Popular methods: DBSCAN, DENCLUE.
- **Grid-based methods**: the feature space is **divided** into **cells** independent of the distribution of the input objects, on which all of the operations for clustering are performed.

  Popular methods: STING, CLIQUE.
- **Model-based clustering**: data are considered as coming from a **mixture** of underlying **probability distributions**, each one linked to a cluster.

  The learning algorithm tries to find these underlying distributions so it can compute the probability of each instance belonging to each cluster, and assign instances accordingly.
- **Graph-based methods**: data are transformed into a **graph representation**: instances are vertices and **edges are weighted** based on the similarity of the instances they link.

An algorithm finds a **optimal partition** of the graph based on an objective function to determine what edges of the graph to cut.

Popular methods: HCS, DTG, CLICK.

- **Other techniques**: combinatorial search, ANNs, kernel-based methods, sequential clustering, ...

Many clustering techniques are based in the application of two measures:

- A **similarity measure** between instances.
  - Usually some kind of **distance metric**.
  - Used to determine if a example should be included in a cluster.
- A **clustering criterion** defining when a **partition** into clusters is **better** than other partition.
- A clustering algorithm uses a similarity measure in order to get a partition that minimizes the clustering criterion.

5.2.1 k-means clustering

The problem: we have a **dataset** D containing n **instances** in Euclidean space.

- Each **instance** x_j is a **vector** $[x_{j1}, x_{j2}, \ldots, x_{jd}]$ of d **real values**.
- Instances are to be split into k **clusters** $C_1, C_2, \ldots, C_k$ (with k provided by the user) such that $C_i \subset D$ and $C_i \cap C_j = \emptyset$ for $1 \leq i, j \leq n$.
- Each cluster is represented by its **centroid** μ_i, the **center point** of the cluster.
- **Similarity measure**: the **euclidean distance** between each object and the centroid of its cluster.

$$d(x_j, \mu_i) = \sqrt{\sum_{k=1}^{d} (x_{jk} - \mu_{ik})^2}$$

- **Clustering criterion**: within-cluster variation, the **sum of squared errors** between all objects in C_i and the centroid μ_i, for all clusters:

$$E = \sum_{i=1}^{k} \sum_{x_j \in C_i} \|x_j - \mu_i\|^2 = \sum_{i=1}^{k} \sum_{x_j \in C_i} \sum_{k=1}^{d} (x_{jk} - \mu_{ik})^2$$

Ideally, we could try to solve this as an **optimization problem** in which the clustering criterion is minimized.

- **Goal**: to find the **partition** that **minimizes** $\boldsymbol{E}$.
- **Problem**: it is computationally very **costly** (time $\mathcal{O}(n^{d\,k+1}\log n)$).
- **Solution**: **greedy** approaches $\Rightarrow$ k-means clustering.

k-means clustering defines the centroid of a cluster as the **mean value of the points** (instances) **within that cluster**.

$$\mu_i = \frac{1}{|C_i|}\sum_{x_j \in C_i} x_j$$

The **algorithm** works as follows:

1. **Initialization**
   - Choose a **value** for $\boldsymbol{k}$.
   - Choose $\boldsymbol{k}$ **random** points in the feature space as **centroids** $\boldsymbol{\mu_i}$.
2. **Learning**

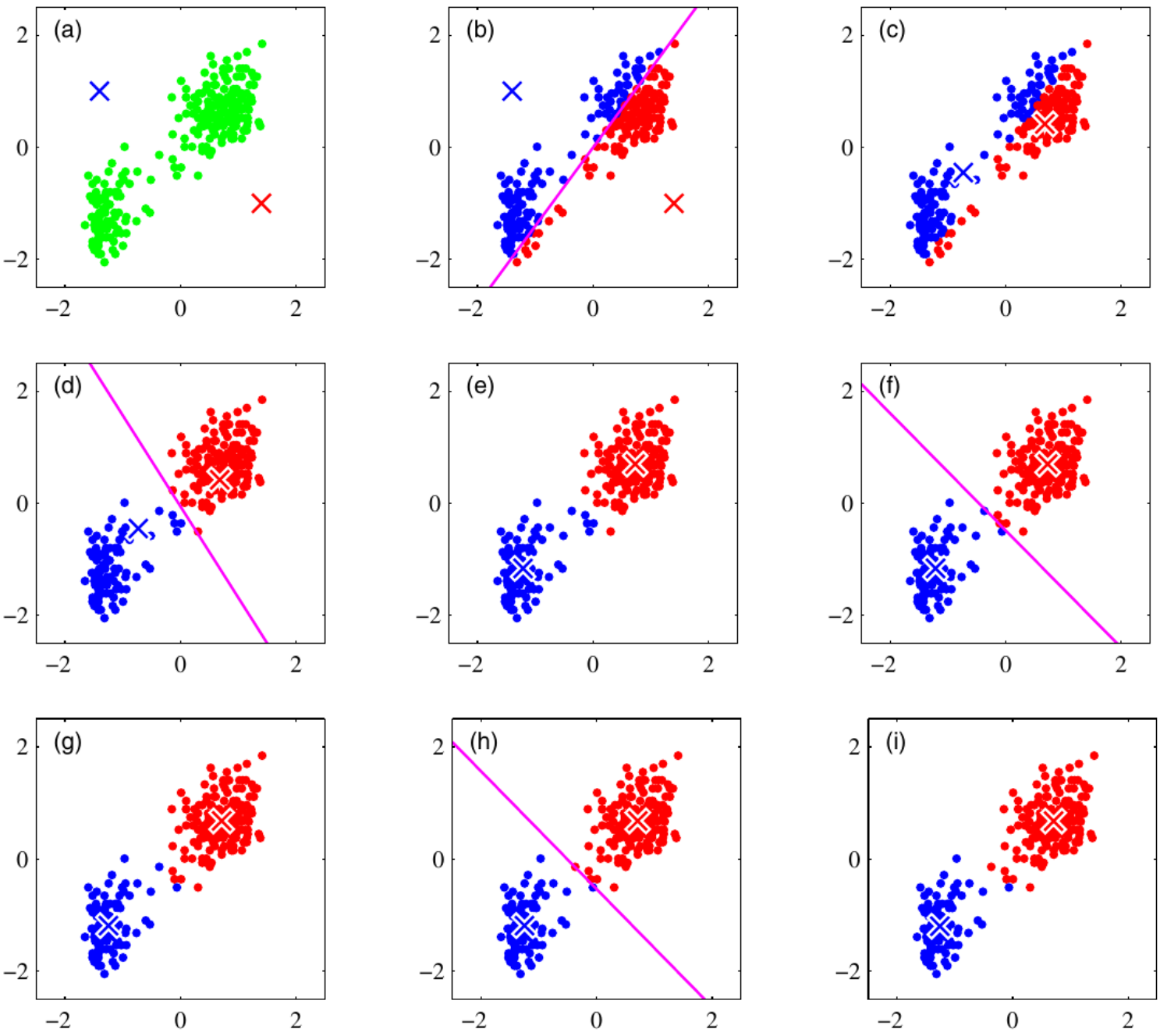
   Repeat:
   - For each **instance** $\boldsymbol{x_j}$:
     - compute the **distance** to each cluster **centroid**.
     - assign the instance to the **cluster** $\boldsymbol{C_i}$ with the **nearest centroid**.
   - For each **cluster** $\boldsymbol{C_i}$:
     - **update** the **centroid** $\boldsymbol{\mu_i}$ to the **mean** value of the points within the cluster.

   Until there is **no change** in the clustering.
3. **Usage**
   - For each **test point**:
     - compute the **distance** to each cluster **centroid**.
     - assign the **instance** to the **cluster** $\boldsymbol{C_i}$ with the **nearest centroid**.

Steps 1 and 2 can**not** guarantee an **optimal solution**.

- They often get stuck in **local optima**.
- They are run **several times** $\Rightarrow$ partition with the lowest E is chosen.

Example:



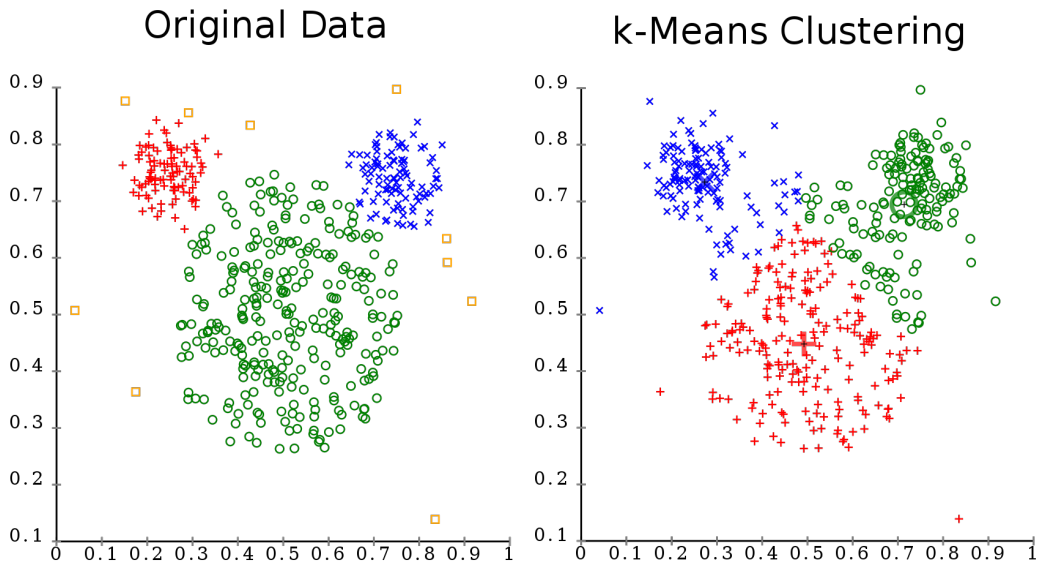
Source: "Pattern Recognition and Machine Learning". C.M. Bishop.

Pros:

- **Simple** and easy to understand.
- Relatively **scalable**: time **complexity** $\mathcal{O}(nkt)$, where t is the total number of iterations, and $k \ll n$, $t \ll n$.
- Good results if clusters are **well separated**, or have **convex** shapes.

Cons:

- The **quality** of the results is dependent on the **value of k**.
  - Too low and some clusters will be grouped together.
  - Too high and some clusters will be split.
- Sensitive to **noise**.
  - The mean value of the points within the cluster is sensitive to outliers.
- Sensitive to the choice of **initial centroids**.
  - Some initial centroids will lead to better solutions that others.
- Not good with clusters with **non-convex** shapes, or **very different sizes**.



Source: wikipedia

There are several **variants** of the k-means method.

- They can differ in the selection of the **initial centroids**, the **clustering criterion**, and the strategies for calculating **cluster means**.
- **k-modes** is a variation of k-means allowing the clustering of **nominal data** by replacing the means of clusters with **modes**.
- **k-medoids** is a variation of k-means in which:
  - The **centroid** μ_i of each cluster C_i is one of the **instances** $x_j \in C_i$,
  - the one with the **lowest sum of distances** to all other members of the same cluster.

$$\mu_i = \arg\min_{x_j} \sum_{x_k \in C_i} d(x_j, x_k)$$

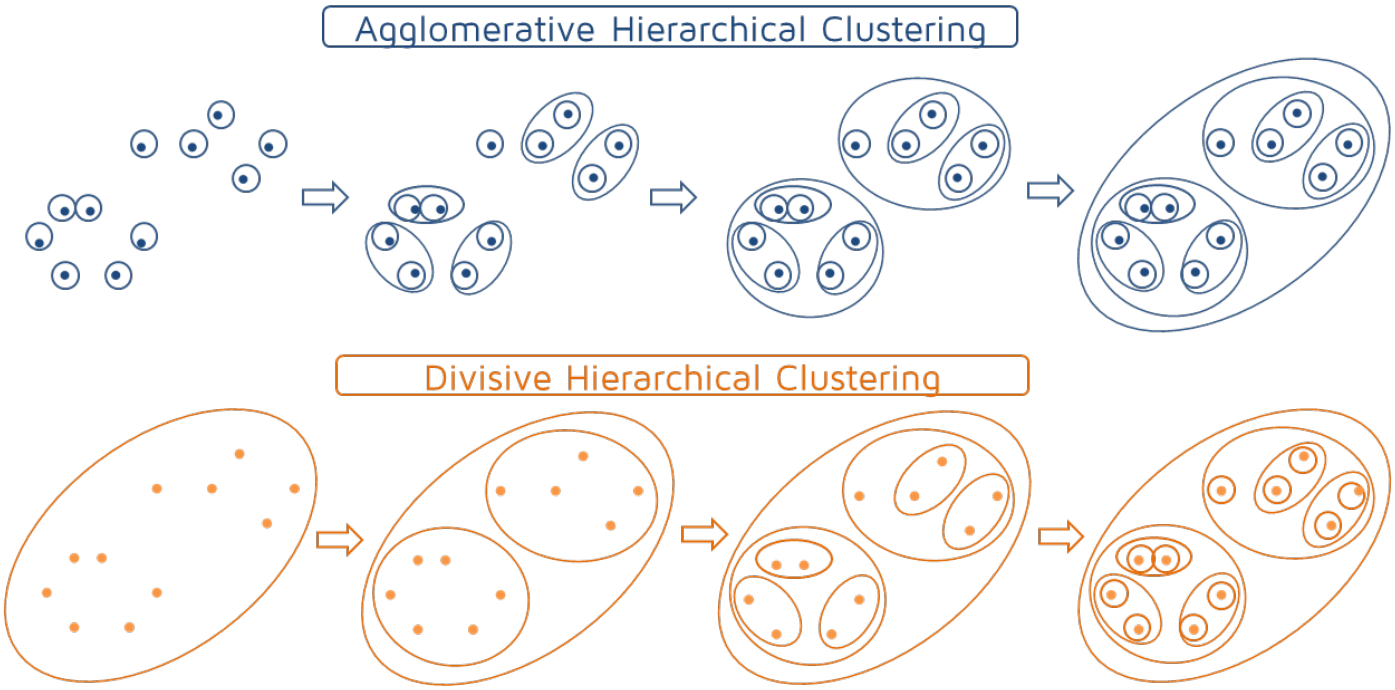
5.2.2 Hierarchical clustering

A **hierarchical clustering method** works by grouping data objects into a hierarchy or **tree of clusters**.

- Clusters at each level of the hierarchy are **merges** of clusters at lower levels.
- At the **root** a single cluster contains **all data**.
- At the **lowest level**, each cluster contains a **single instance**.

Two main classes of hierarchical clustering methods:

- **Agglomerative methods**: bottom-up.
  - they start with each instance forming a **separate cluster**, and
  - **merge** the two clusters with the **smallest intergroup dissimilarity** in each iteration,
  - until all data is **merged** into a **single group**.
- **Divisive methods**: top-down.
  - they start all the objects in the **same cluster**, and
  - iteratively **splits** the existing cluster with the **largest in-group dissimilarity**,
  - until **each instance** is itself **a cluster**.



Source: pinecone.io

- Both paradigms produce **trees** with $\boldsymbol{n-1}$ **levels**, n being the size of the dataset.

Each level of the hierarchy represents a **particular grouping** of the data into **disjoint clusters** of observations.

The entire **hierarchy** represents an **ordered sequence** of such **groupings**.

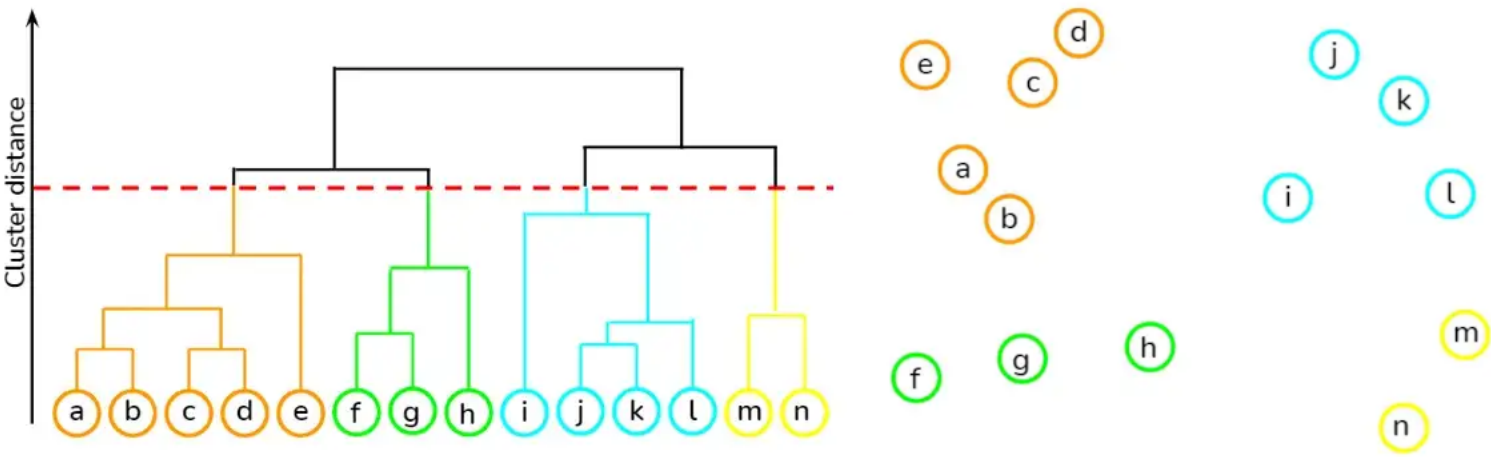
A **dendrogram** is a **tree representation** for hierarchical clustering in which:

- The **height** of each node is **proportional** to the value of the **intergroup dissimilarity** between its two children.
- **Terminal nodes** representing individual observations are all plotted at **zero height**.

The user can **cut** horizontally the **dendrogram** at the level which they decide represents a "natural clustering"

- i.e. intragroup similarity and intergroup dissimilarity are high enough among clusters at that level.

Example of **dendrogram**:



Source: Towards data science

Since **agglomerative clustering** has been more studied than divisive clustering we are going to focus on the former.

An **algorithm** for agglomerative clustering:

1. We start with $\boldsymbol{n}$ **clusters**, one per instance, at **level** $\mathbf{0}$.
2. Repeat:
   - Find the two clusters C_i and C_j with the **biggest similarity**.
   - Create a **new cluster** $C_l = C_i \cup C_j$.
   - C_l is the **parent** of C_i and C_j.
   - Mark C_i and C_j as **unavailable** for **merging**.
   - Mark C_l as **available** for merging.

   until **all data** is in the **same cluster** $\Rightarrow$ the dendrogram **root**.

Temporal **complexity** is $\mathcal{O}(n^3)$,

- $\mathcal{O}(n^2)$ to pick the two most similar clusters.
- $\mathcal{O}(n)$ iterations.

but **can be reduced** to $\mathcal{O}(n^2 \log n)$.

One important part of the algorithm are the **similarity measures** available to decide what clusters to merge, also called **linkage functions**:

- **Single linkage**: the smallest pointwise distance between instances from each cluster.

$$L_{single}(C_i, C_j) = \min_{x \in C_i, x' \in C_j} dis(x, x')$$

- **Complete linkage**: the largest pointwise distance.

$$L_{complete}(C_i, C_j) = \max_{x \in C_i, x' \in C_j} dis(x, x')$$

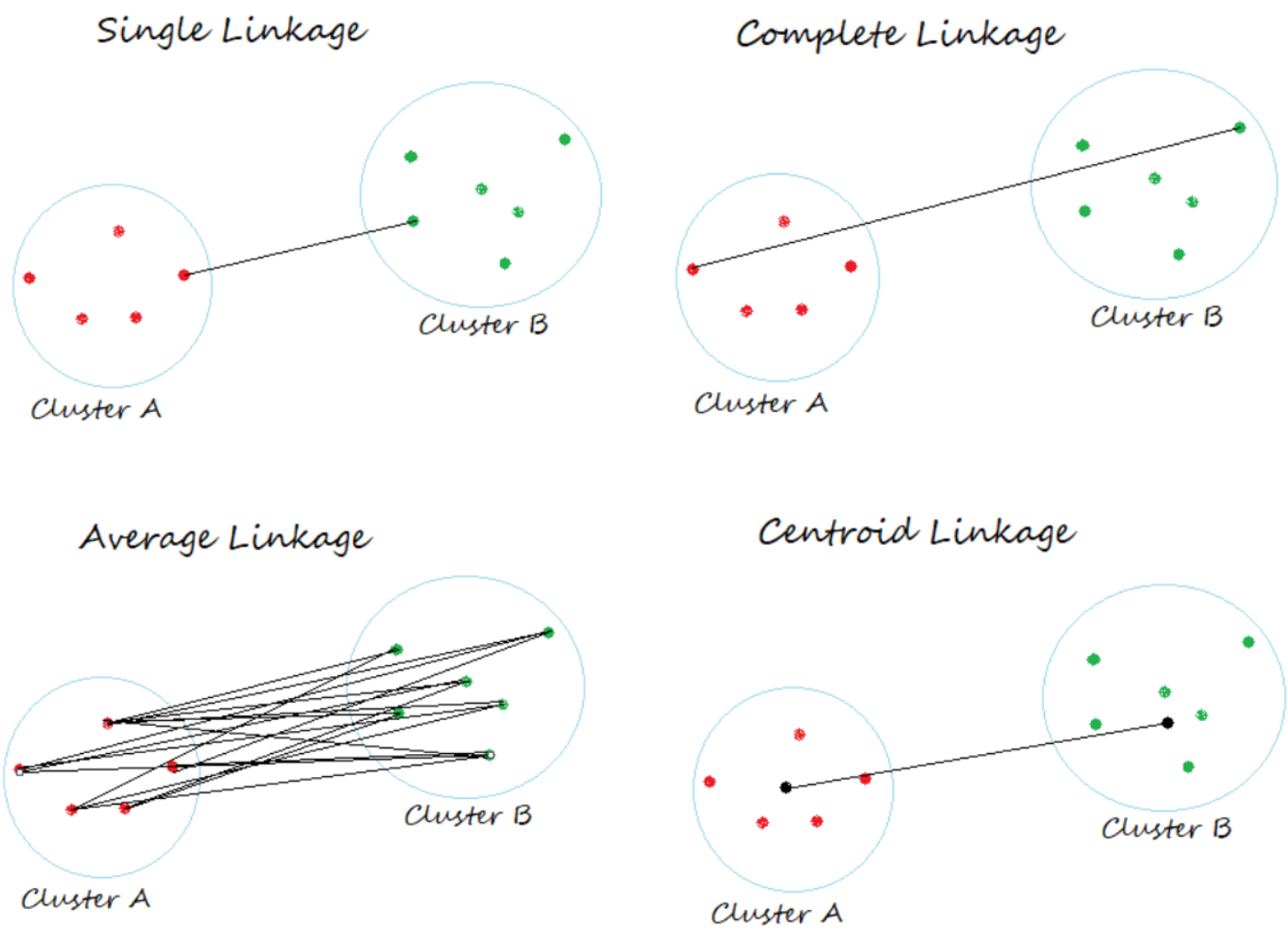
- **Average Linkage**: the average pointwise distance.

$$L_{average}(C_i, C_j) = \frac{\sum_{x \in C_i, x' \in C_j} dis(x, x')}{|C_i|\,|C_j|}$$

- **Centroid linkage**: the point distance between the cluster means.

$$L_{centroid}(C_i, C_j) = dis\left(\frac{1}{|C_i|}\sum_{x \in C_i} x, \frac{1}{|C_j|}\sum_{x' \in C_j} x'\right)$$

where $dis(x, x')$ is a distance metric between points x, x'.



Source: Analytics Vidhya

Single and complete linkage represent **extreme cases**:

- **Single linkage** ⇒ clusters with potentially **very different** observations.
- **Complete linkage** ⇒ compact clusters in which all members are **very similar**.
- Both are **sensitive to outliers**.

Average and centroid linkage attemp to produce clusters:

- with **high similarity** between instances **in a cluster**,
- relatively **low similarity between clusters**,
- representing a **middle ground** between single and complete linkage.

We now go back to hierarchical clustering in general.

Pros:

- There is no need to specify the **number of clusters** in advance.
- The hierarchical structure maps nicely onto human intuition $\Rightarrow$ clustering procedure and results are **easy to understand**.
  - Good for **data visualization**.
  - We can see how data points interact at many levels.
- Hierarchical clustering can detect more **complex-shaped groupings** than k-means $\Rightarrow$ it works better with clusters with different shapes and number of instances.
- **Several** different **clusterings** can be obtained with different **levels of granularity**, cutting at different heights of the dendrogram.
- It can be used to detect **outliers** $\Rightarrow$ single-instance clusters with high between-cluster dissimilarity (distance) with other clusters.

Cons:

- Analysis and partitioning of dendrograms usually **requires human intervention**:
  - Sometimes requires application or domain-specific knowledge to decide for a given partition and to make sense of the results.
  - Interpretation of results can be very subjective.
- **Computationally complex** procedure $\Rightarrow$ difficult to escalate.
  - Huge problem.

5.2.3 Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

A **density-based** procedure.

- It **groups together** data points (instances) that are **closely packed** (in high-density regions with many close neighbours),
- Marks as **outliers isolated points** in low-density regions (with far away nearest neighbors)

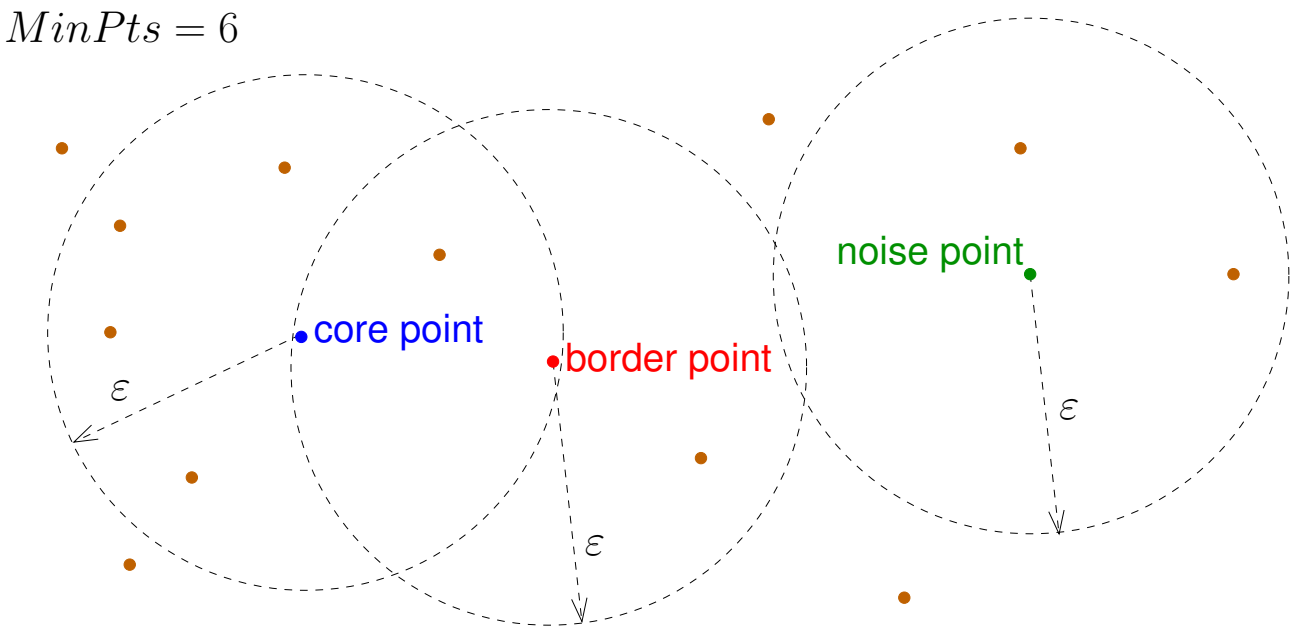
DBSCAN clusters close points based on a **distance threshold** and a **minimum number of points**. Key parameters:

- ε = maximum distance between two data points in a cluster.
- $\boldsymbol{MinPts}$ = minimal density (number of points in a region of radius ε).

Data points can be divided in three categories:

- **Core points**: the ones in the **interior** of a density-based **cluster**.

  An instance is a core point if there are at least $MinPts$ points within ε distance of it (including itself).
- **Border points**: not core points, but falling within the **neighborhood** of one or more **core points** $\Rightarrow$ also part of a cluster.
- **Noise points**: neither a core point nor a border point.



DBSCAN builds a cluster by finding a **core point** and **adding neighboring** core and border points until no more are to be found.

Algorithm:

1. We start with n data points (**instances**), all marked as **unvisited**.
2. Repeat:
   (a) Randomly select an **unvisited point** $\boldsymbol{p}$ and mark it as **visited**.
   (b) if p is a **core point**:
   - Create a new cluster $C = \{p\}$.
   - Let N be the set of points within ε distance from p.
   - For each $p' \in N$:
     - if p' is **unvisited**:
       * mark p' as **visited**.
       * if p' is a **core point**, add the set of points within ε distance of p' to N.
     - if p' is not a member of any cluster, add p' to C.
   - Add C to the list of clusters.

   else mark p as a **noise point**.

   until no point is **unvisited**.

A pretty good illustration of how DBSCAN works here, by Naftali Harris.

- E.g., choose the button "Smiley Face" and values for $MinPts$ and ε.

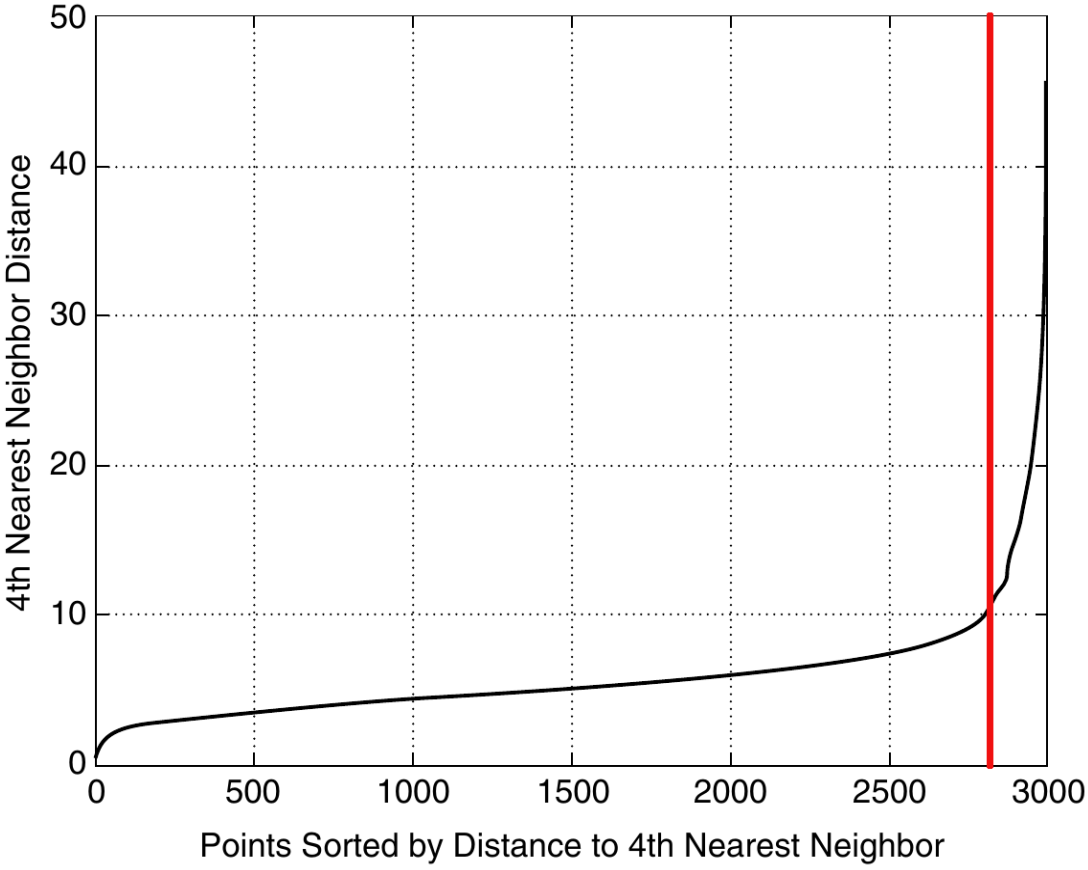
Time **complexity**: given a dataset D of n instances.

- Worst-case scenario: $\mathcal{O}(n^2)$
- For low dimensional data: $\mathcal{O}(n \log n)$

How to **find the values** for the **parameters** of the algorithm:

- $\boldsymbol{MinPts}$:
  - Original paper, $MinPts = 4$, for 2-dimensional data.
  - Heuristic: $MinPts = 2d$ (d being the number of features)
    Larger values may be necessary for:
    * very large data,
    * noisy data,
    * data that contains many duplicates.

- ε: we use a $\boldsymbol{k}$**-distance graph**.
  1. For each data point we compute the longest distance to any of its $k = MinPts - 1$ nearest neighbors.
  2. Those distances are plotted in a graph, ordered from the smallest to the highest value (or viceversa).
  3. ε will be the distance where a "elbow" is found.
     Largest k-distances correspond to outliers.



Source: "Introduction to Data Mining, 2nd Ed". Tan, Steinbach, Karpatne & Kumar. Pearson. 2019.

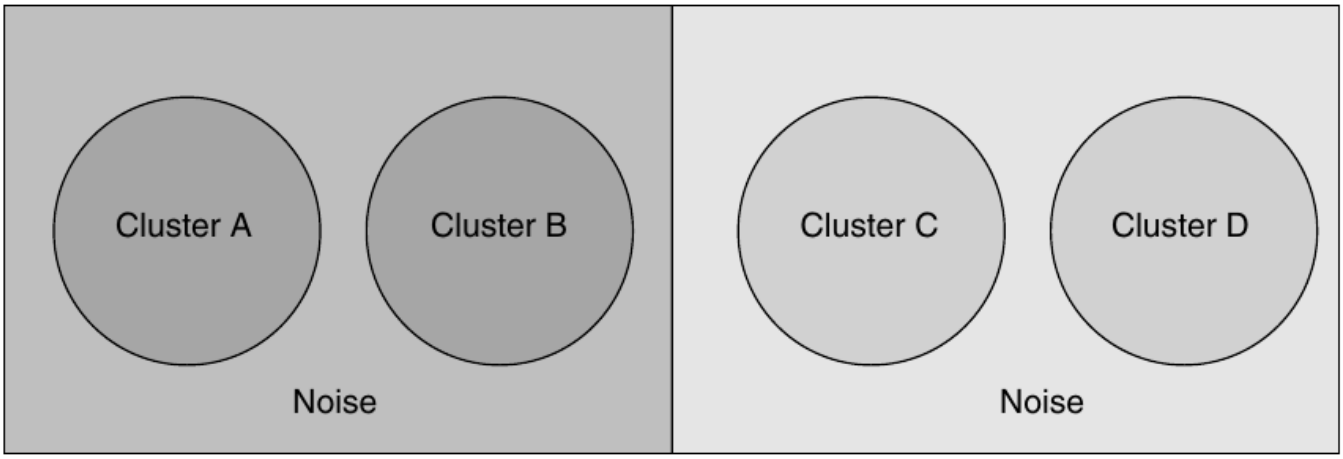
Pros:

- Relatively **resistant to noise**.
- Can handle clusters of **arbitrary shapes and sizes**.
- Automatically marks as **outliers** points in low-density regions.
- Not necessary to **specify in advance** the number of expected clusters.
- Computationally **inexpensive** with low-dimensional data $\Rightarrow$ good scalability.

Cons:

- DBSCAN has trouble with **high-dimensional data**:
  - Distance metrics are rendered almost useless (curse of dimensionality). Little difference in the distances between different pairs of points.
- Necessary to find ε and $MinPts$ values.
- Algorithm **not entirely deterministic**.
  - **Border points** that are reachable from more than one cluster can be part of either one,
  - depending on the **order** in which clusters are processed.
- DBSCAN can have trouble if the **density of clusters** varies widely.

  E.g., in the following figure different grey tones signal different data point densities.



Source: "Introduction to Data Mining, 2nd Ed". Tan, Steinbach, Karpatne & Kumar. Pearson. 2019.

The noise around A and B has the same density than clusters C and D.

- Any values for ε and $MinPts$ that find clusters A and B among the surrounding noise will consider C and D as noise too.
- Any values for ε and $MinPts$ that find clusters C and D, will consider A, B and their surroundings as a single cluster.

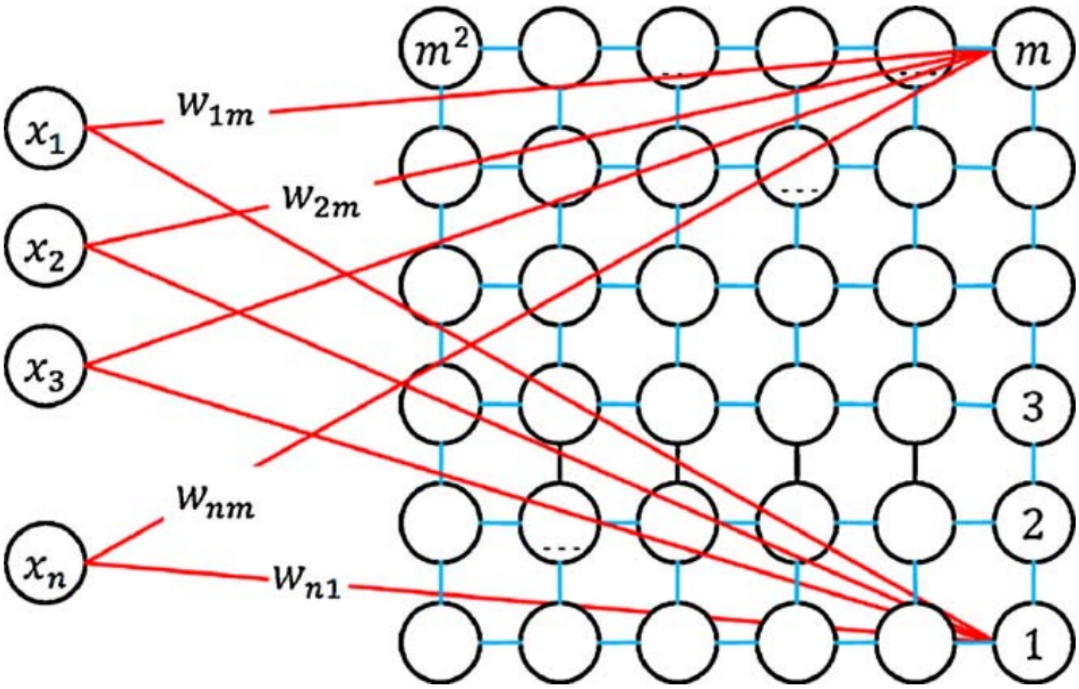
5.3 Unsupervised Neural Networks

5.3.1 Self-Organizing Maps (SOM)

Also called **Self-Organizing Feature Maps**, **Kohonen Maps** or **Kohonen Networks**, the latter two due to their inventor, Teuvo Kohonen.

A SOM consists of two layers:

- An **input** layer.
- A **output layer**, also called **map layer** or **competition layer**, in which a single neuron, called the winning neuron or **Best Matching Unit (BMU)**, is activated when the network receives an input.
  - Neurons in the output layer are arranged in a d-**dimensional grid**.
  - Each neuron is activated by **similar inputs**:
    - **Similarity** is **measured** as distance between the input vectors.
    - Given an input vector x, the **BMU** is the neuron whose weights w_i^T are **more similar** to x (w is the weight matrix).
  - Inputs activating **neighboring neurons** in the output layer are **neighbouring regions** of the input space.



(2D SOM: blue lines represent neighboring units, **not** connections)
Source: Ghaseminezhad & Karami. "A novel self-organizing map (SOM) neural network for discrete groups of data clustering". App. Soft Computing.

The **goal** is to categorize or group **similar instances**:

- **Similar instances** will activate the **same neurons**.
- Each neuron **specializes** in certain (similar) inputs.
- Each group of instances is represented by a **prototype** (the weight vector of their BMU).
  - In clustering, the group of instances is a **cluster** and the **prototype** is the **centroid**.
- Once the network is trained, prototypes are used to **classify** new inputs.

Algorithm:

1. **Initialization**
   (a) Choose a **size** and the number of **dimensions** $\boldsymbol{d}$ for the map.
   (b) Initialize **weights**:
      - Choose random values ensuring all are different, or
      - set the weight values to increase in the direction of the first d principal components of the dataset.
2. **Learning**

   Repeat (outer loop - training epochs):
   - For each **data point** x (inner loop - training instances):
     (a) **Select the BMU** $\boldsymbol{n_b}$ using the minimum euclidean distance between the weights and the input.
     $$n_b = \min_i d(x, w_i^T) = \min_i \|x - w_i^T\| = \min_i \sqrt{\sum_{j=1}^{n} (x_j - w_{ji})^2}$$
     (b) Update the **weight vector** of the **BMU**:
     $$w_{n_b}^T = w_{n_b}^T + \eta(t)(x - w_{n_b}^T)$$
     where $\eta(t)$ is the **learning rate** at instant t.
     (c) Update the **weight vectors** of all **other neurons**:
     $$w_i^T = w_i^T + \eta_n(t)\, h(n_b, t)\, (x - w_i^T), \quad \forall i \neq n_b$$
     where $\eta_n(t)$ and $h(n_b, t)$ are the **learning rate** for neighboring units and the **neighborhood function** at instant t.
     E.g., $h(n_b, t) = 1$ for neighbors and $h(n_b, t) = 0$ for non-neighbors.

(d) Adjust both **learning rates** and **neighbourhood function**:

$$\xi(t+1) = \alpha\, \xi(t)^{k/k_{max}}, \quad \xi(t) \in \{\eta(t), \eta_n(t), h(n_b, t)\}$$

where $0 \leq \alpha \leq 1$ controls the **speed of decrease**, k is the current **epoch** and k_{max} is when we want to stop.

Until the network (weights) stops changing or the maximum number of epochs k_{max} is exceeded.

3. **Usage**
   - For each **test input**:
     - Select the **BMU** n_b using the minimum euclidean distance between the weights and the input.

$$n_b = \min_i d(x, w_i^T)$$

We start with **unordered** neurons in the output layer, with random weights.

Then we start feeding inputs and **updating the weights** of the corresponding BMUs.

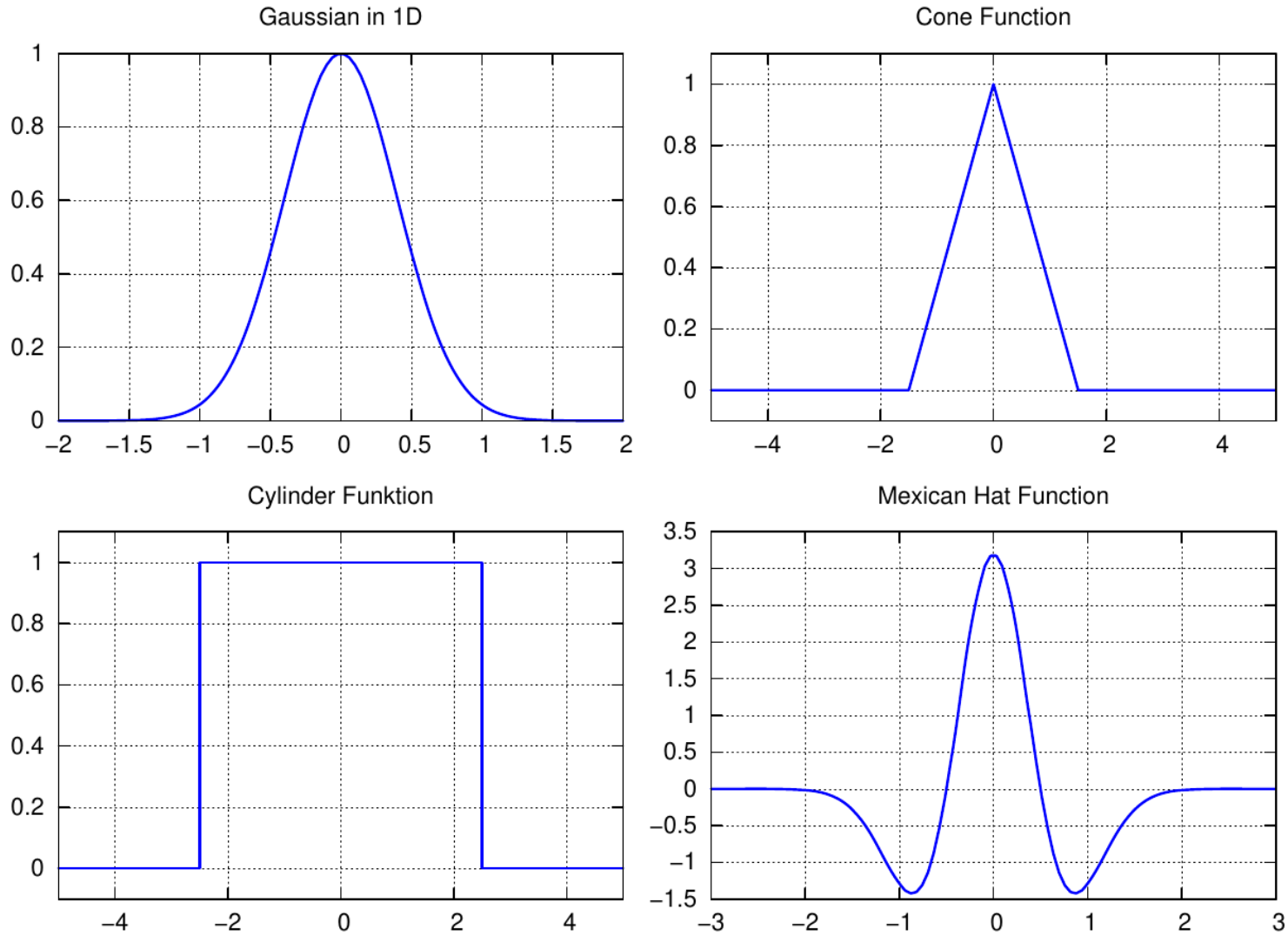
- That makes the **weights** of the BMUs **more similar** to the **inputs**.
- The **units** in the output layer start **reordering** themselves.
  - The weight vectors of units with similar activating inputs become more similar $\Rightarrow$ the units become neighbors.
  - Each time the weights of a BMU are updated, the weights of its neighbors are also updated in the same direction.
  - In clustering, the weight vectors are centroids moving to their final destinations.
- The **neighborhood size** of a BMU is also updated in training:
  - **At the beginning** neighborhoods are **large**,
    - so that we get the rough ordering of the network right.
  - **Later** the size of the neighborhood is **reduced**,
    - so that we can fine-tune the individual local regions of the network.
  - The **size** of the neighbourhood is **reduced** as the network adapts.
- The process typically continues until **convergence**.
  - If the dataset is large we may only use part of it.
  - If the dataset is small, instances x will be used more than once.

- The BMU of an input x can change, the same way a data point can switch clusters in k-means.

Distance measures: though the previous algorithm is based on the euclidean instance, other distance measures can be (and have been) used.

Neighborhood functions: Based on the distance between the weight vectors of the neurons. Several popular options.

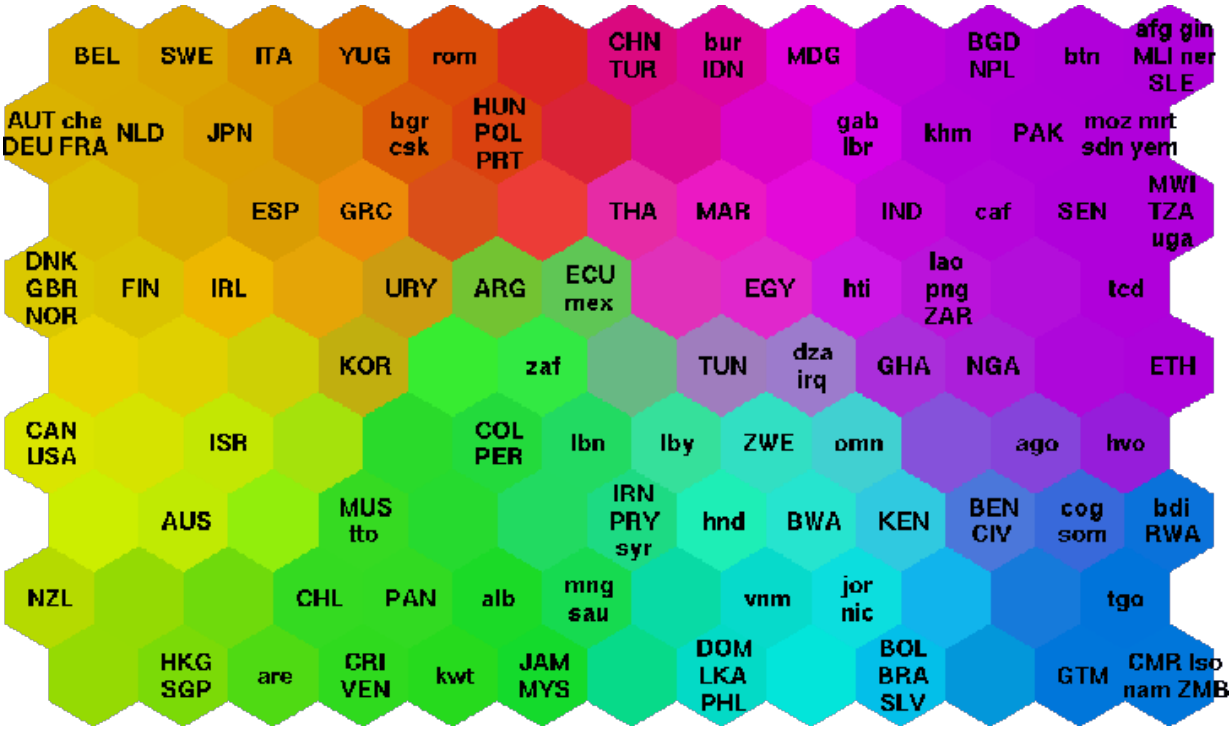
- **Mexican hat function** is the most popular.
- **Gaussian density function**.
- **Cone function**.
- **Step function**, also called cylinder function.

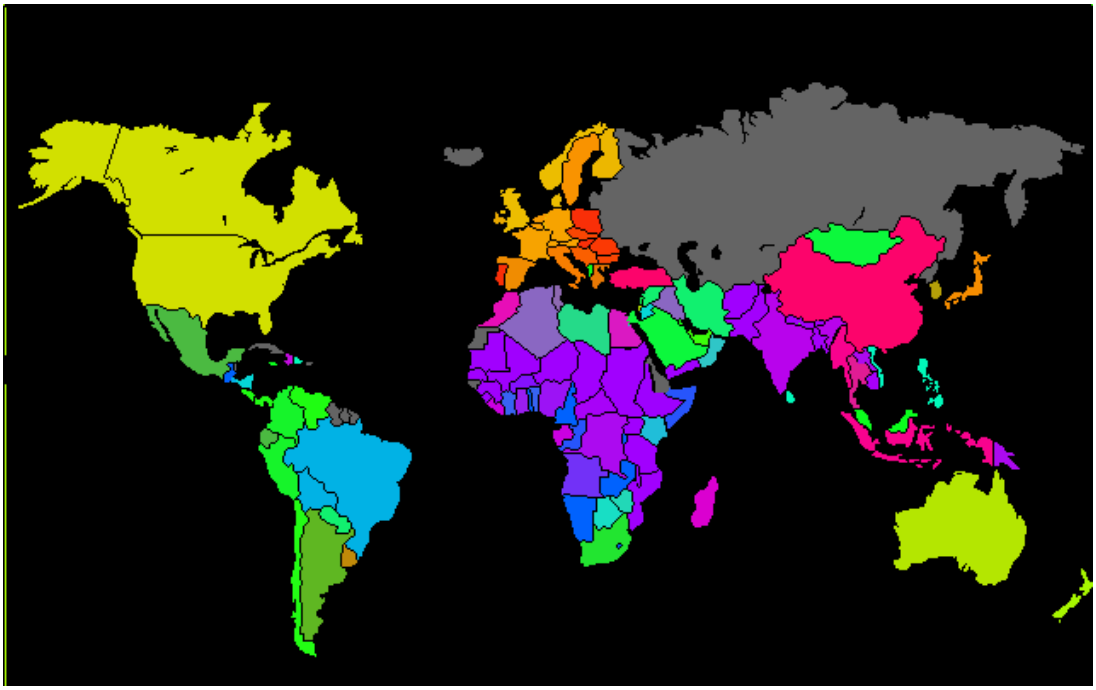


Source: "A Brief Introduction to Neural Networks". David Kriesel.

Example of the application of SOM: World Poverty Map (1992)

- Countries described by 39 indicators of quality-of-life factors: state of health, nutrition, educational services, ...
- Countries that had similar values of the indicators found a place near each other on the map.
- Clusters on the map automatically encoded with different bright colors.
  - Each country is assigned a color.
  - That color describes its poverty type in relation to other countries.





5.3.2 Adaptive Resonance Theory (ART)

Human memory is capable of learning new concepts and habilities without forgetting old ones.

Neural networks tend to "forget" past generalizations if we retrain them.

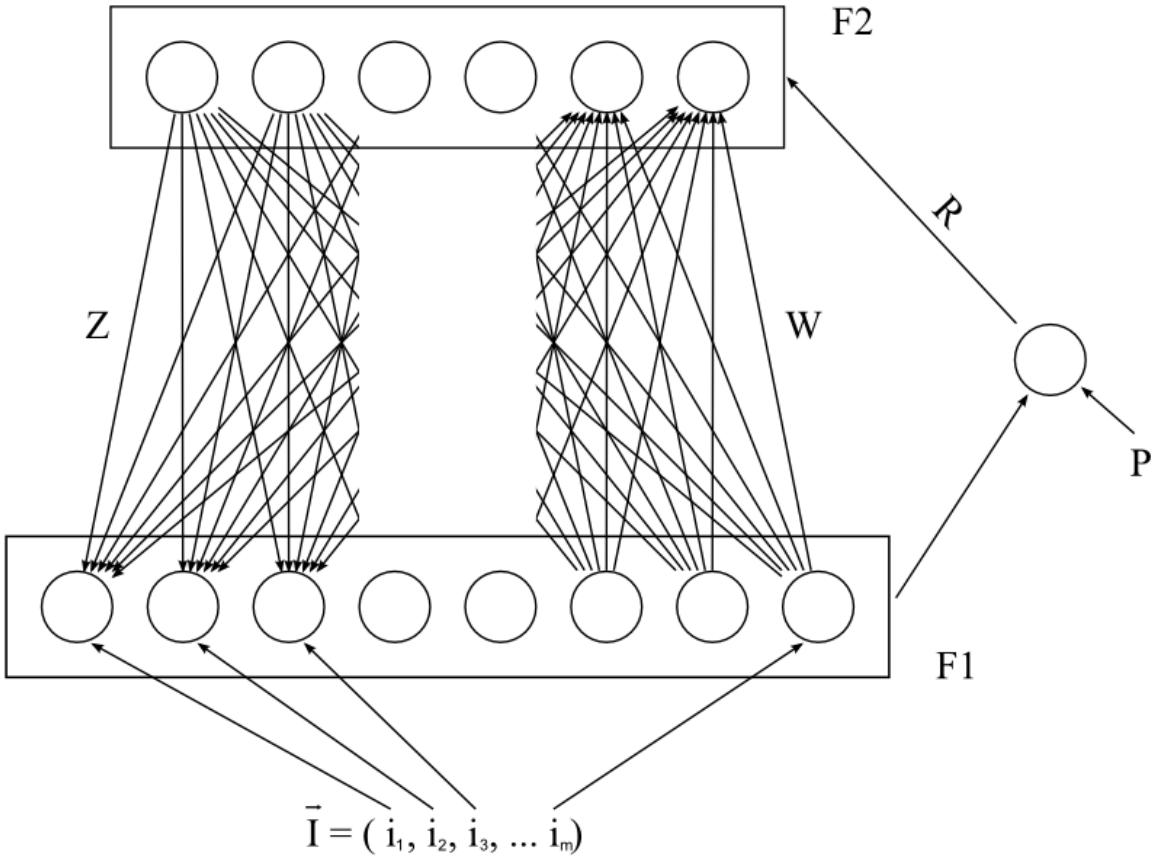
The goal of **Adaptive Resonance Theory** is to build ANNs capable of learning from new information without forgetting what was learned before.

- An attempt to solve the **stability-plasticity dilemma**.
  - Learning requires **plasticity** for the integration of **new knowledge**,
  - but also **stability** to prevent the forgetting of **previous knowledge**.
- ART networks group inputs by similarity in **clusters**.
  - A family of clustering algorithms.
- Unlike SOM, in which the units of the competition layer are fixed, ART can add new neurons (clusters) according to the input.

Like SOM, an ART network consist of **two layers**:

- An **input layer** $\boldsymbol{F1}$: receives the input values, and determines if there is **resonance** (see below).
- An **output layer** $\boldsymbol{F2}$: each **unit** represents a **cluster**.
  - One neuron is activated by an input $\Rightarrow$ cluster to which it belongs.
  - All neurons compete to be the one that activates.
  - The **winning neuron** inhibits the others in $F2$.
- Both **layers** are **fully connected** in both directions:
  - **Forward connections** with weights $\boldsymbol{W}$, used to determine the **winning neuron** in $F2$.
  - **Backward connections** with weights $\boldsymbol{Z}$, used as the **prototype** of the cluster.
- In addition there is a control signal called **vigilance theshold** $\boldsymbol{\rho}$.
  - $0 \leq \rho \leq 1$.
  - ρ regulates how **similar** the **input** and the **prototype** of the winning neuron have to be so the input is accepted in the cluster, i.e. they are in **resonance**:

* If they are in **resonance**, the input is accepted and the weights of the winning neuron are modified so it is more similar to the input.
* If they are **not in resonance**, a **reset signal** R is turned on and the network searches for a new winning neuron in the remaining units of $F2$.
* If no winning neuron in resonance with the input is found, a **new unit** is added to $F2$ to cluster that input.



Source: wikipedia (Author: Christoph Müller)

Algorithm:

- We are going to discuss the algorithm for **ART1**, the first developed ART network for **binary inputs**.
- There is a more complex version for **continuous inputs**, **ART2**.
- During the running of the network there are not differentiated training and usage phases.

  The network just memorizes new inputs and generates new clusters if they are needed.

1. **Initialization**
   (a) **Weights** W and Z are initialized:
   $$W_{ij}(0) = \frac{1}{1+N} \qquad Z_{ji}(0) = 1 \qquad 1 \leq i \leq N,\ 1 \leq j \leq M$$
   - N is the number of **neurons** in $F1$,
   - M is the number of **neurons** in $F2$,
   - 0 is the initial iteration.

   (b) A value for the **vigilance threshold** ρ is chosen, with $0 \leq \rho \leq 1$.
2. **Learning**
   - For each input instance x:

   (a) The **activation** of each unit in $F2$ is computed.
   $$\hat{y}_j = \sum_{i=1}^{N} W_{ij}(t)\, x_i$$
   (b) The neuron k with the biggest activation is selected as the **winning neuron**.
   $$k = \arg\max_j \sum_{i=1}^{N} W_{ij}(t)\, x_i$$
   (c) **Vigilance test**: $F1$ checks if the **similarity** between the input x and the backward weights of the winning neuron k is bigger than the **vigilance threshold**:
   $$\frac{Z_k(t) \cdot x}{x \cdot x} = \frac{\sum_{i=1}^{N} Z_{ki}(t)\, x_i}{\sum_{i=1}^{N} x_i^2} > \rho$$
   If the condition **does not** hold:
   - **Reset** signal R disables unit k.
   - Go to step 2.b.
     (a winning neuron will be looked up among the remaining ones)

   (d) The **weights** of neuron k in W and Z are **updated**.
   For all $0 \leq i \leq N$:
   $$Z_{ki}(t+1) = Z_{ki}(t)\, x_i \qquad W_{ik}(t+1) = \frac{Z_{ki}(t)\, x_i}{0.5 + \sum_{j=1}^{N} Z_{kj}(t)\, x_j}$$
   (e) Enable all neurons in $F2$.

If the network is incapable of finding any unit on $F2$ for which the vigilance test holds, i.e it is in **resonance** with the input:

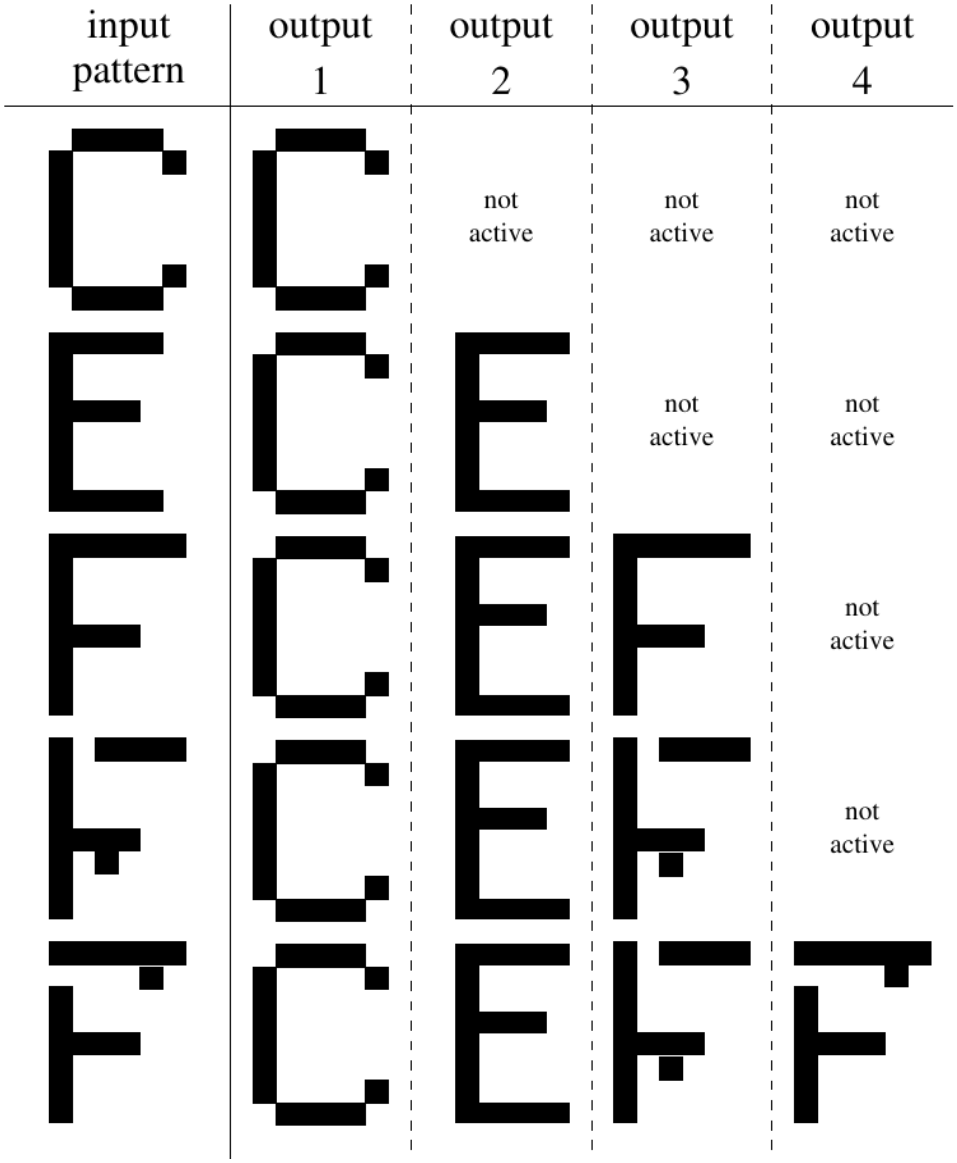
- The network **saturates**.
- A **new unit** is added to $F2$ whose weights W and Z are initialized to the values of x.
  - New **winning unit**.
  - The new neuron is in resonance with the input, due to its weights.
- The network learns a new instance (plasticity), different from the rest, without affecting existing clusters (stability).

The **vigilance threshold** $\boldsymbol{\rho}$ sets the **granularity** of clustering:

- Defines the **minimal strength** of the **similarity** between prototype and input to ensure **resonance**.
- **Low values** of ρ (near 0):
  - Few **large clusters**.
  - **Misclassification** more likely.
- **High values** of ρ (near 1):
  - Many **small clusters**.
  - Higher **precision**.

Example: Character recognition.

- ART1 network.
- Input: letters as small black squares in a 8x8 grid. Binary inputs.
- The sequence of input instances are represented at the leftmost column of the figure, from top to bottom.
- The stored prototypes (Z) are represented right of the sequence of inputs.
- Each row represents the input and state of the network at an time instant.



Source: "An introduction to neural networks". Kröse, van der Smagt.